



Code Quest · AI Agent

What Are AI Agents?

AI agents are autonomous or semi-autonomous systems that can perceive their environment, make decisions, and take actions to achieve specific goals. They typically combine large language models (LLMs) with additional capabilities like:

- Memory and context management
- Tool usage (APIs, data sources, applications)
- Action execution capabilities

The Potential of AI Agents as LLMs Approach AGI

As LLMs continue to evolve toward capabilities that resemble aspects of Artificial General Intelligence (AGI), AI agents built upon these models could transform our relationship with technology. These advanced agents would move beyond today's limited domain expertise toward systems that can operate across multiple domains with minimal human guidance. By combining increasingly sophisticated reasoning, planning, and learning capabilities with persistent memory and specialized tools, future agents might effectively serve as autonomous personal assistants capable of handling complex, open-ended tasks that today require significant human intervention.

The societal implications of such capable agents would be profound. In professional contexts, they could function as collaborative partners that augment human creativity and productivity rather than simply executing commands. For example, a researcher might delegate literature reviews, experimental design iterations, and data analysis to



an agent that maintains awareness of the project's evolving goals. In personal contexts, agents could manage increasingly complex aspects of daily life, from optimizing financial decisions to coordinating healthcare management across providers. The economic value created by these systems could be substantial, potentially democratizing access to capabilities previously available only to those who could afford human specialists.

However, this evolution raises critical questions about agency, accountability, and control. As agents gain autonomy in decision-making, the boundaries of human oversight become less clear. Who bears responsibility when an agent makes a consequential error? How do we ensure these systems remain aligned with human values as their capabilities expand? The development of AI agents approaching AGI capabilities will require careful consideration of these governance challenges, potentially necessitating new regulatory frameworks and technical safeguards that balance innovation with appropriate constraints. The transition from today's narrow agents to more general systems may ultimately depend as much on solving these social questions as on technical breakthroughs.

Main Task: Create an AI Agent that can play Blackjack

Create a **static webpage** (HTML, CSS, and JavaScript) that implements a Blackjack-playing AI agent. The user provides their API key by uploading a `.env` file — the key is read in-memory only and never stored or transmitted beyond the API call. The agent reads the current game state (player hand, dealer up card), calls an LLM for a recommendation, and executes the action.

When compared to previous assignments, the main challenge here revolves around extracting the eLLe's recommendation (e.g., hit, stand) from its response. One approach is to use a keyword search to find the recommendation. This is problematic because a response can contain multiple actions. For example, "Given your hand and

the dealers up card, it is not recommended that you hit. You should stand." This text contains both "hit" and "stand" which will cause problems if you are using a keyword search.

It is important to develop code that provides output behind-the-scenes (e.g., to the console) so you can see what is happening. If you inspect the Blackjack Agent shown below, you will see the messages I asked Claude Sonnet to create to help me understand the interactions with the eLLe. This will give you a hint to how I solved the problem.

It is important to test your AI Agent extensively (i.e., play lots of Black Jack hands). For each hand you play, you should read the AI Analysis and compare that to the recommendation. Check to make sure these are consistent with one another. If they are click the Execute Recommendation button to make sure the action taken is, in fact, the action recommended. And lastly, make sure the Balance is being updated correctly.

BLACKJACK WITH AI AGENTS

ABOUT

MODEL FAMILY:

OpenAI (12 models)

MODEL:

GPT-4o (gpt-4o-2024-11-20)

BET AMOUNT:

10

Balance: \$1000

PLAY

REFRESH MODELS

DEALER



Score: 10

PLAYER



Score: 14**Current Bet: \$10**

Using a Reference Implementation

Before you start building, set up a `temp/` folder inside your project repository containing a working example of a static webpage that interacts with an LLM via an uploaded `.env` file. Claude Code will use this as a reference — not as part of your final project.

The reference implementation shows Claude Code exactly how you want things done: how to parse the `.env` file for the API key (in-memory only), how to structure the `fetch()` call to the LLM API, and how to handle responses and errors. Your Blackjack agent will follow those same patterns.

What to Tell Claude Code

Include something like the following when you start your Claude Code session:

Prompt for Claude Code PLAINTEXT

```
1 The temp/ folder contains a working example of a static webpage
2 that interacts with an LLM via an uploaded .env file. Use it as
3 a reference for:
4   - How to parse a .env file for the API key (in-memory only)
5   - The fetch() call structure for the LLM API
6   - Error handling patterns for failed API requests
7
8 Do NOT include the temp/ folder in the final build or deployment.
9
10 Build a static Blackjack AI agent page that:
11   - Accepts an API key via .env file upload
12   - Implements a full Blackjack game (deck, dealing, scoring)
13   - Calls the LLM to recommend hit/stand given the current hand
14   - Returns a structured JSON response so the action can be
15     extracted reliably (avoids keyword-search ambiguity)
16   - Logs key interactions to the console for debugging
17
```

Show less

Stretch Challenge: Add 2 or more enhancements to the webpage.

Some ideas for enhancements are listed below.

1. **Strategy Visualization** - Display a visual representation of the optimal decision matrix alongside the AI's recommendation, showing why the agent chose a particular move based on probabilities.

2. **Performance Analytics** - Add metrics tracking that shows win rate, bankroll growth/decline, and decision quality over time.
3. **Explainability Controls** - Add toggles for different levels of explanation detail from the agent (basic recommendation, statistical reasoning, or in-depth analysis).
4. **Risk Tolerance Settings** - Allow users to set different risk profiles and see how recommendations change.